Prompt Engineering

Prompts as Specifications, Not Just Inputs

Matt Redmond

11 Feb 2026

Today's Journey

1. **What Happens When You Prompt**
   - Tokens, conditioning, and why prompts steer behavior
2. **Prompts as Behavioral Specifications**
   - Why prompts are code, not prose
3. **Anatomy of a Prompt**
   - System prompts, user prompts, and their roles
4. **Prompt Techniques**
   - Few-shot, chain-of-thought, reasoning models, structured output
5. **Evaluation, Reflection, and Guardrails**
   - LLM-as-judge, self-critique, guardrail prompts
6. **Prompt Versioning and Testing**
   - Treating prompts like code: version control, regression testing

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

Your Tokens Are the Model's Entire Reality

Recall from Unit 2: an LLM is a next-token predictor trained on language.

When you send a prompt, here is what happens:

1. Your text is split into **tokens**—the model never sees "words" or "meaning" directly.
2. Each token is mapped to an embedding—a point in a high-dimensional space.
3. The transformer layers process these embeddings, building up rich representations of the input.
4. The model generates the *next* token based on the *entire* processed context.

The Key Point

The model has no memory, no hidden understanding, and no intent of its own. Your token sequence is the **only** input that determines what it does next. There is **no other way** to interact with a model beyond inserting tokens into its context.

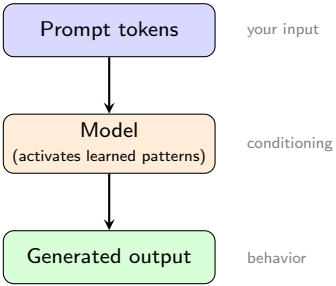
Prompting Is Conditioning, Not Asking

A prompt doesn't "tell" the model what to do—it **conditions** it.

What "conditioning" means:

- During training, the model learned patterns: *given input like X, produce output like Y.*
- Your prompt activates the patterns most consistent with the token sequence you provided.
- Different prompts activate different patterns—even in the same model.

Think of it as setting up the **launch conditions** for generation: small changes in the prompt can send the model on a completely different trajectory.



Implication

Prompting is not really just asking—it's **configuring**. That's why we treat prompts as specifications.

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

What Is a Prompt, Really?

A prompt is not just a question—it's a behavioral specification.

The Key Insight

A prompt defines *what* the model should do, *how* it should do it, and *what form* the output should take. It's the primary interface between your intent and the model's behavior.

A prompt specifies:

- **Role:** Who the model should act as
- **Task:** What the model should accomplish
- **Constraints:** Boundaries on behavior and output
- **Format:** The structure of the expected response
- **Examples:** Demonstrations of desired behavior

Think of prompts as *specifications*, not *requests*.

Why "Prompt Engineering" Matters

The same model can behave very differently depending on the prompt.

Vague prompt

User: "Help me with this customer complaint: [complaint text]"

Model: *Three paragraphs of generic customer-service advice... no classification, no actionable next step.*

Precise prompt

System: Classify complaints as `billing` | `technical` | `other`. Rate severity 1–5. Suggest a resolution in $\leq$2 sentences. Respond in JSON.

User: "[complaint text]"

Model: `{"category":"billing", "severity":4, "resolution":"Issue refund..."}`

Prompt quality determines:

- **Accuracy:** Does the model produce correct outputs?
- **Consistency:** Does the model behave the same way across inputs?
- **Safety:** Does the model avoid harmful or off-topic responses?
- **Cost:** Longer prompts cost more; efficient prompts save money

The Uncomfortable Truth

Most "model failures" are actually prompt failures. Before blaming the model, audit the prompt for contradictions or missing info.

Prompts Are "Code" For Models

If prompts define system behavior, they deserve the same rigor as code:

Code practices:

- Version control
- Code review
- Unit testing
- Regression testing
- Documentation

Prompt practices:

- Prompt versioning
- Peer review of prompts
- Evaluation on golden datasets
- Regression testing on changes
- Comments explaining intent

Principle

Version and test prompts like code. A prompt change can break your system just like a code change.

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

System Prompts vs. User Prompts

Recall from Unit 2: the chat API has distinct message roles.

System prompt:

- Sets the model's persona, role, and constraints
- Persists across the conversation
- Written by the *developer*
- Examples: "You are a helpful legal assistant. Always cite sources."

User prompt:

- Contains the specific request or question
- Changes each turn
- Written by the *end user* (or constructed by your application)
- Examples: "Summarize this contract in 3 bullet points."

Design Principle

Put *stable* behavior in the system prompt; put *variable* content in the user prompt. This separation enables prompt caching and cleaner architecture.

What Goes in the System Prompt?

A well-structured system prompt typically includes:

1. **Identity / Role:** Who the model is and its expertise
2. **Task description:** The high-level objective
3. **Behavioral constraints:** What the model should and should *not* do
4. **Output format:** JSON, markdown, specific schema (can use Structured Output)
5. **Examples:** (Optional) Few-shot demonstrations
6. **Edge-case handling:** What to do when uncertain

Common Pitfall

Overloading the system prompt with too many instructions. If the prompt is too long, the model may ignore parts of it (recall "lost in the middle" from Unit 2).

Example: Annotated System Prompt

Scenario: A support-ticket classifier for a SaaS company.

[1] You are a senior support agent for Acme Cloud with expertise in billing, accounts, and technical troubleshooting.

[2] For each support ticket, classify it into a category and draft a concise, empathetic customer-facing reply.

[3] Rules: Never promise refunds or credits—escalate those to a human agent. Do not speculate about outages; link to status.acme.io. Keep replies under 150 words.

[4] Respond in JSON:
`{"category":"billing|technical|account|other", "priority":"low|medium|high", "draft_reply":"..."}`

[5] Example ticket: "I was charged twice this month."
→ `{"category":"billing", "priority":"high", "draft_reply":"I apologize for the duplicate charge. I have flagged your account for review by our billing team..."}`

[6] If a ticket is ambiguous, ask one clarifying question before classifying. If the ticket is not in English, reply in the customer's language.

Key:

[1] Identity / Role

[2] Task description

[3] Behavioral constraints

[4] Output format

[5] Few-shot example

[6] Edge-case handling

Maps to the six components on the previous slide.

Takeaway

~120 words—short enough for the model to follow reliably, specific enough to constrain behavior.

What Goes in the User Prompt?

The user prompt carries the variable, per-request content:

1. **The specific query or instruction:** What the user actually wants *right now*
2. **Runtime data:** Documents, code snippets, or context to reason over.
3. **Input parameters:** Values your application injects at request time

Good user prompt content:

- "Summarize this article: [article text]"
- "Classify this ticket: [ticket body]"
- "Review this code: [diff]"

Bad user prompt content:

- Role definitions (belongs in system)
- Output format specs (belongs in system)
- Behavioral guardrails (belongs in system)

Rule of Thumb

If the content would be the same across every request, it belongs in the system prompt. If it changes per request, it belongs in the user prompt.

How They Work Together: API Structure

In the chat API, system and user prompts are separate messages with distinct roles:

stable (developer)

`role: "system"` You are a legal assistant. Always cite sources. Respond in JSON {...}.

variable (per turn)

`role: "user"` Summarize this contract in 3 bullet points: [contract text]

`role: "assistant"` {"bullets": ["...", "...", "..."]}

`role: "user"` Now identify any liability clauses.

Why?

- **Prompt caching:** The longest possible common prefix is cached and reused across requests, reducing latency and cost
- **Testability:** You can test system and user prompts independently

Prompt Templates: Separating Logic from Content

In production, prompts are often implemented as "templates filled at runtime":

Python — template with placeholders

```
SYSTEM = """
You are a {role} for {company}.
Classify each {item_type} into: {categories}.
Respond in JSON.
"""
prompt = SYSTEM.format(role="support agent",
    company="Acme Cloud", item_type="ticket",
    categories="billing, technical, account, other")
```

Benefits of templating:

- **Reusability:** Same prompt structure, different inputs
- **Testability:** Test the template with known inputs
- **Separation of concerns:** Prompt logic vs. runtime data
- **Caching:** Shared prefix enables prompt caching

Practical Tips

Keep your prompt templates separate from application code, as standalone files. It can be helpful to build them up such that you can leave comments in line.

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

Zero-Shot vs. Few-Shot Prompting

Zero-shot: Give the model a task with no examples.

Few-shot: Provide examples of desired input–output pairs in the prompt.

Zero-shot

System: Classify the sentiment of the review as `positive`, `negative`, or `mixed`.

User: "The battery lasts forever but the screen is impossible to read in sunlight."

Model: `positive` ✗
(Ambiguous—model guesses without calibration)

Few-shot (3 examples)

System: Classify the sentiment of the review as `positive`, `negative`, or `mixed`.

"Amazing sound quality!" → `positive`
"Broke after one week." → `negative`
"Great camera, terrible UI." → `mixed`

User: "The battery lasts forever but the screen is impossible to read in sunlight."

Model: `mixed` ✓

The term "few-shot prompting" was popularized by Brown et al. (2020), who showed that GPT-3 could perform tasks with just a handful of in-context examples—no gradient updates required ("Language Models are Few-Shot Learners").

When to Use Few-Shot

- Task requires a specific format the model doesn't default to
- Classification with domain-specific labels
- Ambiguous tasks where examples clarify intent

Few-Shot Best Practices

Getting the most out of few-shot examples:

1. **Diversity:** Cover the range of expected inputs (easy, hard, edge cases)
2. **Consistency:** All examples should follow the same format
3. **Ordering:** Place most representative examples first and last (recall "lost in the middle")
4. **Quantity:** 3–5 examples is often sufficient; more is not always better
5. **Quality:** Goes without saying: incorrect examples poison the conditioning

Watch Out

Few-shot examples consume tokens. Balance the quality improvement against cost and context-window usage.

Chain-of-Thought (CoT) Prompting

Idea: Encourage the model to reason step by step before answering.

Recall from Unit 2: adding "Let's think step by step" improves reasoning performance.

Why does CoT work?

- LLMs generate tokens sequentially—reasoning tokens create "working memory"
- Intermediate steps reduce the difficulty of each individual prediction
- The model can "show its work" and catch errors along the way

Without CoT

User: "If 5 machines take 5 minutes to make 5 widgets, how long would 100 machines take to make 100 widgets?"

Model: "100 minutes." ✗

With CoT

User: "...same question...
Let's think step by step."

Model: "5 machines make 5 widgets in 5 min $\Rightarrow$ each machine makes 1 widget in 5 min. 100 machines each make 1 widget in 5 min $\Rightarrow$ **5 minutes**." ✓

Key Insight

CoT doesn't add new capabilities—it gives the model "scratch space." Reasoning tokens become part of the context for subsequent tokens, reducing the difficulty of each prediction.

CoT Variants

Several flavors of chain-of-thought:

- **Zero-shot CoT:** "Let's think step by step" (no examples)
- **Few-shot CoT:** Provide examples with explicit reasoning chains
- **Self-consistency:** Generate multiple CoT paths, take the majority answer
- **Tree-of-thought:** Explore branching reasoning paths

When to use CoT:

- Math and logic problems
- Multi-step reasoning tasks
- Tasks requiring planning or decomposition
- *Not* simple factual recall or classification

Trade-off

CoT increases output token count (and cost/latency). Use it when the reasoning improvement justifies the overhead.

Reasoning Models: CoT "For Free"

Key development: Modern *reasoning models* are trained on reasoning traces (often via reinforcement learning), so they produce chain-of-thought *natively*—no special prompting required.

Traditional CoT

- You add "think step by step"
- Reasoning tokens appear in the visible output
- You pay for reasoning tokens in the response
- Works with any instruction-tuned model

Reasoning Models

- Model reasons automatically
- Thinking may be hidden (e.g. OpenAI o-series) or visible (e.g. DeepSeek-R1)
- Often billed separately as "reasoning tokens", but sometimes just "output tokens"
- Trained via RL on reasoning traces

Examples: OpenAI o1/o3, DeepSeek-R1, Claude with extended thinking, Gemini 2.0 Flash Thinking

Practical Implication

If you are already using a reasoning model, explicit CoT prompting is often **redundant**—and can even hurt by over-constraining the model's internal reasoning. Match your prompting strategy to the model class: use CoT prompts for standard models, and let reasoning models think on their own.

Structured Output Prompting

Goal: Get the model to produce output in a specific, parseable format.

Approaches (from weakest to strongest guarantees):

1. **Natural language instruction:** "Respond in JSON format"
2. **Few-shot with formatted examples:** Show the desired structure
3. **Schema in prompt:** Provide the exact JSON schema or Pydantic model
4. **Constrained decoding:** API-level enforcement (e.g., OpenAI structured outputs)

Why Structured Output Matters

Structured output is what enables LLMs to be integrated into software systems. Without parseable output, you can't build reliable pipelines.

Structured Output Best Practices

Getting reliable structured output:

- **Be explicit:** Specify the schema in the prompt, not just "return JSON"
- **Use constrained decoding when available:** API-level guarantees beat hope
- **Validate output:** Always parse and validate; retry on failure
- **Keep schemas simple:** Complex nested structures increase error rates
- **Include field descriptions:** Help the model understand what each field means

Practical Tip

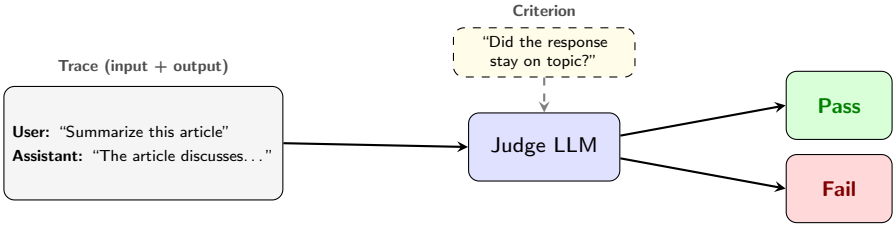
Libraries like Pydantic (Python) or Zod (TypeScript) let you define schemas that serve as both prompt documentation and output validation. Use them.

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

LLM-as-Judge

Idea: Pass an execution trace into a separate LLM that acts as a **classifier** for target behaviors.



Tips for effective LLM judges:

- Judge **straightforward boolean properties** with low ambiguity
  (e.g. "Did the response include a citation?" not "Is this response good?")
- One criterion per judge call—compose multiple judges for complex rubrics
- Cheaper and faster than human eval; calibrate against human labels regularly

Caution

LLM judges have biases (verbosity bias, position bias, self-preference). Calibrate against human judgments.

Self-Critique and Reflection

Pattern: Ask the model to review and improve its own output.

The reflection loop:

1. **Generate:** Model produces an initial response
2. **Critique:** Model evaluates its own response against criteria
3. **Revise:** Model improves the response based on its critique
4. (Optionally repeat)

When to use reflection:

- Tasks where first-pass quality is insufficient
- Complex outputs (long documents, code, analysis)
- When latency budget allows multiple LLM calls

Trade-off

Reflection improves quality but multiplies cost and latency (2–3× per iteration).

Guardrail Prompts

You can use prompts to enforce safety and policy constraints:

- **Input guardrails:** Classify user input before processing
  - Is this on-topic? Is it a prompt injection attempt?
- **Output guardrails:** Check model output before returning to user
  - Does the response contain PII? Is it factually grounded?

Defense in Depth

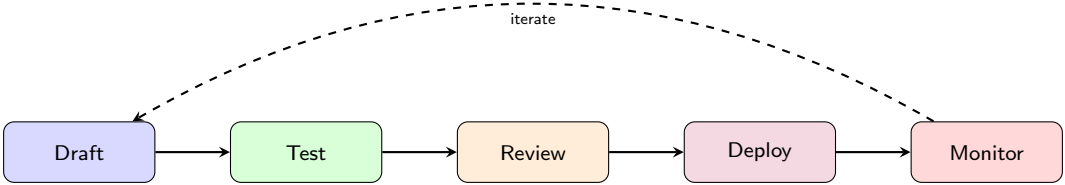
Guardrails are not foolproof—use multiple layers (prompt-level, API-level, application-level) for robust safety. Providers also often have their own guardrails in place at the API level.

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

The Prompt Lifecycle

Prompts evolve over time—manage them like software artifacts:



- **Draft:** Write the initial prompt
- **Test:** Evaluate against your golden dataset
- **Review:** Peer review prompt changes
- **Deploy:** Ship with confidence
- **Monitor:** Track performance in production, iterate

Evaluating Prompt Changes

How do you know if a prompt change is an improvement?

1. **Define metrics:** What does "better" mean for this prompt?
   - Accuracy, format compliance, latency, cost
2. **Run on golden dataset:** Use the evaluation dataset from Unit 3
3. **Compare A vs. B:** Old prompt vs. new prompt on the same inputs
4. **Check for regressions:** Did the change break any previously passing cases?
5. **Statistical significance:** Is the difference real or just noise? This is REALLY HARD to do well!

Connecting to Unit 3

This is where your golden dataset and evaluation framework pay off. Without them, you're guessing.

Prompt Versioning Strategies

How to track prompt versions:

- **Git:** Store prompts as files, it's by far the easiest.

What to track for each version:

- The prompt text itself
- Model and parameters used (model name, temperature, etc.)
- Evaluation results on the golden dataset (can be metadata)
- Author and rationale for the change

Practical Tip

At minimum, keep your prompts in version control. Even a simple `prompts/` directory with dated files is better than editing prompts inline.

Online Testing Prompts

In production, test prompt variants with real traffic:

A/B testing workflow:

1. Deploy both prompt variants simultaneously
2. Route a percentage of traffic to each
3. Collect metrics (quality scores, user feedback, latency)
4. Analyze results and promote the winner

Considerations:

- Ensure sufficient sample size for statistical significance
- Monitor for unexpected regressions on subpopulations
- Consider cost differences between variants

Outline

1. What Happens When You Prompt
2. Prompts as Behavioral Specifications
3. Anatomy of a Prompt
4. Prompt Techniques
5. Evaluation, Reflection, and Guardrails
6. Prompt Versioning and Testing
7. Conclusion

Key Takeaways

1. **Prompts are specifications, not just inputs**
   - They define the model's behavior, constraints, and output format
2. **Prompts steer the model into specific behaviors via conditioning**
   - The model has no memory, no hidden understanding, and no intent of its own.
3. **Techniques: few-shot, chain-of-thought, structured output**
   - Each has trade-offs in quality, cost, and latency
4. **Evaluation, reflection, and guardrails improve output quality**
   - LLM-as-judge, self-critique, and guardrail prompts
5. **Version and test prompts like code**
   - Golden datasets turn prompt engineering from art into engineering
6. **Much of this pre-supposes *one-shot* generation without access to tools**
   - We'll cover extensions to this framework later!

Further Reading

Course reading:

- Shankar & Husain, Chapter 2 (§2.1–2.4)

Foundational:

- Brown et al. (2020) — "Language Models are Few-Shot Learners" (GPT-3)
- Wei et al. (2022) — "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"
- Wang et al. (2023) — "Self-Consistency Improves Chain of Thought Reasoning in Language Models"
- Zheng et al. (2023) — "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena"

Practical:

- OpenAI Prompt Engineering Guide — `platform.openai.com/docs/guides/prompt-engineering`
- Anthropic Prompt Engineering — `docs.anthropic.com/en/docs/build-with-claude/prompt-engineering`

Thank you!